Efficient and Effective Similar Subtrajectory Search: A Spatial-aware Comprehension Approach

LIWEI DENG, HAO SUN, RUI SUN, University of Electronic Science and Technology of China, China
YAN ZHAO*, Department of Computer Science, Aalborg University, Denmark
HAN SU, School of Computer Science and Engineering and Yangtze Delta Region Institute (Quzhou), University of Electronic Science and Technology of China, China

Although many applications take subtrajectories as basic units for analysis, there is little research on the similar subtrajectory search problem aiming to return a portion of a trajectory (i.e., subtrajectory), which is the most similar to a query trajectory. We find that in some special cases, when grid-based metric is used, this problem can be formulated as a reading comprehension problem, which has been studied extensively in the field of natural language processing (NLP). By this formulation, we can obtain faster models with better performance than existing methods. However, due to the difference between natural language and trajectory (e.g., spatial relationship), it is impossible to directly apply NLP models to this problem. Therefore, we propose a Similar Subtrajectory Search with Graph Neural Networks (3SGNN) framework. This framework contains four modules including a spatial-aware grid embedding module, a trajectory embedding module, a query-context trajectory fusion module and a span prediction module. Specifically, in the spatial-aware grid embedding module, the spatial-based grid adjacency is constructed and delivered to the graph neural network to learn spatial-aware grid embedding. The trajectory embedding module aims to model the sequential information of trajectories. The purpose of the query-context trajectory fusion module is to fuse the information of the query trajectory to each grid of the context trajectories. Finally, the span prediction module aims to predict the start and the end of a subtrajectory for the context trajectory, which is the most similar to the query trajectory. We conduct comprehensive experiments on two real world datasets, where the proposed framework outperforms the state-of-the-art baselines consistently and significantly.

CCS Concepts: • **Computing methodologies → Supervised learning by classification**; **Neural networks**.

Additional Key Words and Phrases: similar subtrajectory search, graph neural networks, reading comprehension

ACM Reference Format:
Liwei Deng, Hao Sun, Rui Sun, Yan Zhao, and Han Su. 2021. Efficient and Effective Similar Subtrajectory Search: A Spatial-aware Comprehension Approach. *ACM Trans. Intell. Syst. Technol.* 1, 1, Article 1 (January 2021), 22 pages. https://doi.org/10.1145/3456723

*Corresponding author: Yan Zhao.

Authors' addresses: Liwei Deng, Hao Sun, Rui Sun, University of Electronic Science and Technology of China, China, deng_liwei@std.uestc.edu.cn,sunhao19990814@gmail.com,sunrui@std.uestc.edu.cn; Yan Zhao, Department of Computer Science, Aalborg University, Denmark, yanz@cs.aau.dk; Han Su, School of Computer Science and Engineering and Yangtze Delta Region Institute (Quzhou), University of Electronic Science and Technology of China, China, hansu@uestc.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2018 Association for Computing Machinery.
2157-6904/2021/1-ART1 $15.00
https://doi.org/10.1145/3456723

1 INTRODUCTION

The prevalence of GPS devices such as smartphones has generated massive spatial trajectory data, which raises many analysis methods of trajectory such as clustering [1, 4, 14] and similarity search [2, 6, 16, 41]. There are lots of studies [4, 7, 14, 15, 17–20, 30, 31, 40, 47, 49–51, 54] which treat subtrajectories, (i.e., a portion of trajectories), as basic units for analysis. For example, Lee et al. [14] present a subtrajectory clustering method which splits the raw trajectories into subtrajectories based on some rules and then groups the splited subtrajectories into clusters. Buchin et al. [4] aim to detect commuting patterns using clustering subtrajectories.

The similar trajectory search (SimTraj) problem aims to find a trajectory from the database, which is the most similar to the query trajectory. However, in some cases, the two trajectories may be dissimilar when the context trajectory is treated as a whole but some portion of context trajectory is similar to the query trajectory. The similar subtrajectory search (SimSub) problem, formally proposed by Wang et al. [31], aims to return a subtrajectory that is the most similar to the query trajectory based on a trajectory similarity measurement such as t2vec [15]. In the SimSub problem, the original trajectory needs to be splited and a subtrajectory which is similar to the query trajectory is returned. While in the SimTraj problem, the query and context trajectory are considered as a whole. Compared with SimTraj, SimSub can capture trajectory similarity in a finer-grained way.

To solve the SimSub problem, Wang et al. [38] propose a novel solution, i.e., RLS, which treats the splitting process as a Markov decision process (MDP) and employs an advanced reinforcement learning technique. Further, they propose a variant RLS, called RLS-Skip, which is more efficient than RLS by reducing the search space during the training and inference process. Although these methods outperform the baselines such as UCR [22, 24, 25] and Spring [27] in terms of effectiveness, they are not efficient enough, which indicates their infeasibility in real similar trajectory search scenarios. We observe that in some special cases, when the similar trajectory measurement is based on spatial grid, the similar subtrajectory search problem can be formulated as a reading comprehension problem, which is a well-studied problem in NLP. After transforming to a reading comprehension problem, the original problem can be solved more efficiently.

In this paper, we focus on the similar subtrajectory search problem where t2vec [16], a grid-based trajectory similarity metric, is used as the trajectory similarity measurement. Then we solve the similar subtrajectory search problem by a reading comprehension paradigm, in which the well-known reading comprehension models such as QANet [46] and BERT [8] can be used here. However, there are some differences between natural language and trajectories. The first difference is that the patterns in trajectories maybe different from those in natural language. For example, in the natural language, the long-term dependency is important because the latter words may have strong correlation with the previous words in a long range. However, in a trajectory, the long-term dependency information is not important. In most case, the later point in trajectory is just strongly related to the previous few points. The second difference which is also the biggest difference is that there is spatial relationship between grids in trajectories while there is no explicit relationship between words in the natural language. Figure 1 shows the spatial relationship in grids, where the whole geometric space is splited into 5×5 grids. It is obvious that the blue grid is closer to the orange one than the green one, which shows that the relations among grids are different in geometric view. Due to the differences between natural language and trajectories, the existing reading comprehension models, e.g., QANet [46], cannot be directly applied to the similar subtrajectory search problem.

To solve the problem and bridge the gaps between natural language and trajectories, we propose novel framework, namely a Similar Subtrajectory Search with Graph Neural Networks (3SGNN),

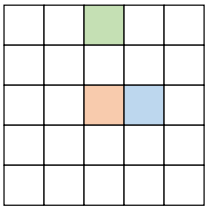


Fig. 1. Spatial proximity example. The whole geometric space is splited into 5×5 grids. It is obvious that the blue grid is nearer to the orange grid than the green grid, which shows that the relations among grids is different in geometric view.

which consists of four modules, i.e., a spatial-aware grid embedding module, a trajectory embedding module, a query-context trajectory fusion module and a span prediction module. Specifically, the spatial-aware grid embedding module is used to model the spatial relationship between grids. In this module, we propose two grid graph construction methods and use graph neural networks to capture the neighboring relationship. In the trajectory embedding module, we explore some sequential models such as QA-block [46] and BiLSTM [3, 28] to model the sequential information in trajectories. In the query-context trajectory fusion module, three different methods, i.e., a naive fusion method and two advanced attention-based fusion methods, are adopted to fuse the query information to each grid of the context trajectory. Finally, the span prediction module is used to predict the start and the end of a subtrajectory of the context trajectory, which is the most similar to the query trajectory.

As demonstrated by the experiments, GBAF, which is derived from our proposed 3SGNN framework, is the most effective and efficient method compared with all the baselines on two real-world datasets. In particular, GBAF outperforms POS-D, which performs best among the traditional methods, by 46% for MR on the Harbin dataset. GBAF outperforms the best (i.e., RLS) among the learnable methods, by 23% in terms of MR. Moreover, GBAF takes the least time for training compared with the trainable methods, i.e, RLS, RLS-Skip, and G-QANet, and it is the most efficient method in the inference phase.

The major contributions of this paper are as follows:

- We propose a 3SGNN framework to solve the SimSub problem by transforming the problem as a reading comprehension problem.
- An advanced spatial-aware embedding technique is proposed to model the spatial relationship between spatial grids. Specifically, two grid graph construction methods are proposed and then the constructed graph is delivered to graph neural networks to model the spatial relationship.
- We also explore two sequential models to capture the sequential information of trajectories. Moreover, two advanced attention-based fusion methods are adopted in the query-context trajectory fusion part.
- Extensive experiments are conducted on two real-world trajectory datasets, i.e., Porto and Harbin. The results show the superiority of the proposed framework both in effectiveness and efficiency compared with the state-of-the-art methods such as RLS, RLS and POS-D.

2 RELATED WORK

2.1 Trajectory Similarity Measurements

The trajectory similarity measurement is a fundamental problem in many trajectory researches such as clustering [1, 14], similar trajectory search [11, 13, 48, 52, 53], and similar subtrajectory

search [38]. With the advent of deep learning, the trajectory similarity measurement can be divided into two groups, i.e., traditional measurements and deep measurements. The traditional trajectory similarity measurements are mostly based on dynamic programming techniques such as dynamic time wrapping (DTW) [45], longest common subsequence (LCSS) [35], edit distance with real penalty (ERP) [5], edit distance on real sequences (EDR) [6], and Frechet [2]. Specifically, these methods usually try to form a pairwise matching the sample points of two trajectories and identify the best alignment using dynamic programming [16]. t2vec [16] is a deep measurement of trajectory similarity which maps the longitude-latitude points to grid index and then adopts a sequence-to-sequence framework with RNN to encode trajectories to a Euclidean space, in which distance metrics of vectors such as L2 distance and L1 distance can be used to measure the similarity of two trajectories. NeuTraj [44] leverages deep metric learning to accelerate computation process of traditional trajectory similarity measurements such as Frechet, ERP and DTW.

2.2 Similar Subtrajectory Search

The SimSub problem is formally proposed by Wang et al. [38] aiming to return a portion of context trajectory which is the most similar to the query trajectory. Similar to trajectory similarity measurements, the solution of SimSub also can be divided into traditional methods and deep methods. SizeS [38] is a traditional method. When computing the similarity between the query trajectory and subtrajectories of context trajectory, subtrajectories that have similar length to the query trajectory will be considered in SizeS. Other traditional methods such as POS and POS-D [38] are all based on splitting strategies. Specifically, POS scans the points of context trajectory and computes the similarity between the query trajectory and the former part before the current scaned point. POS-D is a variant of POS. Whenever a prefix is found to be more similar to the query trajectory than the best subtrajectory known so far, POS-D continues to scan n more points and splits at one of these $n+1$ points. RLS and RLS-Skip [38] are two deep methods to solve the SimSub problem. Inheriting the idea of splitting-based methods, these deep methods treat the splitting process as a MDP and then adopt reinforcement learning to model the MDP.

2.3 Reading Comprehension

The reading comprehension problem [8, 29, 36, 46] is a well-studied problem in NLP. Given a question sentence and a context paragraph, this problem requires to return a portion of context which contains information that can answer the question. The Match-LSTM with Ans-Ptr [37] is to solve the reading comprehension problem by combining Match-LSTM [36] and Pointer-Net [34]. Inspired by this work, a large number of models adopt Pointer-Net [34] to predict the start and the end of the answer in the context paragraph. BiDAF [29] proposes a bidirectional attention mechanism, i.e., context-to-query attention and query-to-context attention, which can improve the performance compared with the one-directional attention model [37, 42]. QANet [46] also adopts the bidirectional attention architecture and proposes an encoder block which we call QA-block for convenience. The QA-block is a totally new sequential model which is not similar to the former models that adopt RNN or the variant of RNN. Only convolutional neural networks and attention mechanism are used in QA-block which can be treated as the predecessor of BERT [8]. BERT, as a pretrained language model, can not only be used to solve the reading comprehension problem but also other problems of natural language such as classification of sentences. Although BERT performs better than QANet in reading comprehension problems, the performance of BERT is mainly dependent on the pretrained process with large corpus. Besides, the pretrained BERT on the trajectory dataset is time-consuming and difficult. QANet is the state-of-the-art model before the emergence of BERT and its variants, e.g., XLNet [43]. Therefore, we choose QANet as one of our baselines in the experiments (Section 6.2).

Table 1. Summary of notation

Notations	Definition
p_i	The longitude-latitude point on the map
g_i	A grid that p_i is located in
T, T_c	An input context trajectory
$\|T\|$	The length of trajectory $\|T\|$
$T[i, j]$	A subtrajectory that starts from the i-th point and ends at j-th point
T_q	An input query trajectory
$\Theta(\cdot, \cdot)$	A trajectory similarity measurement

3 PROBLEM DEFINITION AND PRELIMINARIES

We first introduce preliminaries and then formulate the Similar Subtrajectory Search (SimSub) problem. Table 1 summarizes notation used throughout the paper.

Definition 1. (TRAJECTORY) A trajectory data is the trace of a moving object which can be expressed as a sequence of points, i.e., $T_k =< p_1, p_2, ..., p_n >$, where k indicates the k-th trajectory in dataset D and point $p_i = (lon_i, lat_i)$ represents a longitude-latitude point on the map. The length of trajectory T_k is denoted by $|T_k|$ and the size of dataset D is denoted by $|D|$. When k is clear from the context, we use T to denote T_k.

Definition 2. (SUBTRAJECTORY) Given a trajectory T, (i.e., $|T| = n$, and $1 \leq i \leq j \leq n$), we denote the subtrajectory as $T[i, j]$, which represents the portion of T that starts from the i-th point and ends at the j-th point.

Definition 3. (READING COMPREHENSION PROBLEM) Given a context paragraph with n words $C = \{c_1, c_2, ..., c_n\}$ and a question sentence with m words $Q = \{q_1, q_2, ..., q_m\}$, the reading comprehension problem aims to output the span $S = \{c_i, c_{i+1}, ..., c_{i+j}\}$ from the original paragraph C that can answer the question.

Similar Subtrajectory Search (SimSub) Problem Statement. Given a context trajectory $T =< p_1, p_2, ..., p_n >$ and a query trajectory $T_q =< q_1, q_2, ..., q_m >$, the similar subtrajectory search problem aims to find a subtrajectory of T, denoted by $T[i^*, j^*]$ $(1 \leq i^* \leq j^* \leq n)$, which is the most similar to T_q according to a trajectory similarity measurement $\Theta(\cdot, \cdot)$, i.e., $[i^*, j^*] = argmax_{1 \leq i \leq j \leq n} \Theta(T[i, j], T_q)$.

The SimSub problem can be transformed to a reading comprehension problem if we treat the answer of the question in the paragraph as a special measurement and each word in a paragraph as one trajectory point in a trajectory. The only difference between the two problems is that a trajectory is a sequence of longitude-latitude points with continuous values in the SimSub problem, while the natural language is composed of discrete words in the reading comprehension problem. Therefore, the primary task is to convert the longitude-latitude points to indexes (or called grids) and use the corresponding similarity measurements (called grid-based trajectory similarity metrics) to compute the similarity between two sequences of indexes.

3.1 Trajectory Similarity Measurements

According to the problem statement, SimSub is based on a trajectory similar measurement, which can be formulated as a reading comprehension problem when the grid-based metric is used. To map the longitude-latitude points to grids, we adopt a simple strategy that is used commonly in spatial data analysis, i.e., we partition the space into grids of equal size [9, 16] and treat each grid as a token.

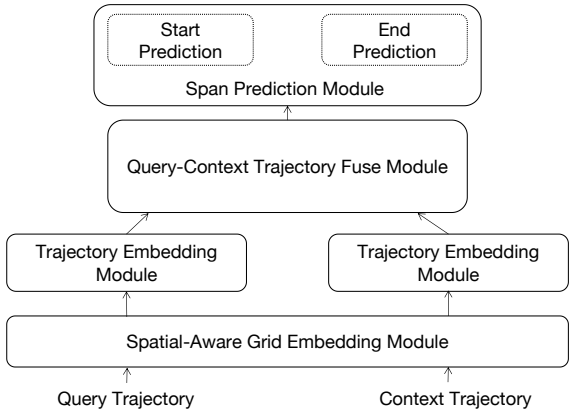


Fig. 2. The proposed framework. There are four modules involved, spacial-aware grid embedding module, trajectory embedding module, query-context trajectory fusion module and span prediction module.

All the longitude-latitude points falling into the same grid are then mapped to the same token. The mapped longitude-latitude trajectory, called grid trajectory, is denoted by $T =< g_1, g_2, ..., g_n >$ [1].

t2vec [16] is a recently proposed grid-based data-driven trajectory similarity method, which adopts a sequence-to-sequence framework of RNN to learn the trajectory representation. In practice, the sampling rate of trajectory often varies across devices due to different devices settings, battery constraints, and communication failures. Even for the same device, the sampling rates may vary. To solve the non-uniform and low sampling rate problem, t2vec uses a randomly dropping trajectory as input and aims to reconstruct the non-dropping trajectory while the pure sequence autoencoder has the same input and output. Moreover, to model the spatial relationship between grids, a spatial proximity aware loss function is used. Compared with the traditional trajectory similarity methods, e.g., LCSS [35], EDR [6], EDwP [26], t2vec can get better performance in self-similarity and cross-similarity comparisons, and find the k-nearest neighbors problems. As a result, t2vec is chosen as the metric to measure the similarity of two trajectories.

4 METHODOLOGY

In this paper, we propose a framework to solve the SimSub problem in which t2vec [16] is used as the trajectory similarity measurement. As shown in Figure 4, the framework consists of four modules, namely spatial-aware grid embedding module, trajectory embedding module, query-context trajectory fusion module and span prediction module. Specifically, in the embedding module, the spatial-based grid adjacency is constructed and delivered to the graph neural networks to learn the spatial-aware grid embedding. The trajectory embedding module aims to model the sequential information of trajectories. For the query-context trajectory fusion module, the purpose is to fuse the information of the query trajectory to each grid of context trajectories. The last span prediction module is used to predict the start and the end of a subtrajectory of the context trajectory, which is the most similar to the query trajectory. In the following, we provide specifics on each module in the framework.

[1]In the follow, trajectory means grid trajectory in the absence of special instructions.

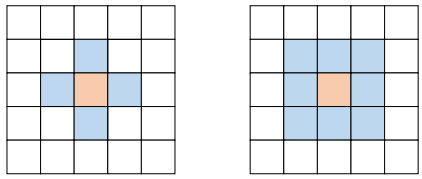


Fig. 3. Construction of grid graph example. The whole space is splited into 5×5 grids. Now we focus on the orange grid and treat it as central grid. We give two ways to define the neighbors. For the left part, we define four neighbors, i.e., the blue grids, which are adjacent to the orange one. For the right part, we define eight neighbors which are around the central grid.

4.1 Spatial-Aware Grid Embedding Module

In the fields such as NLP and recommender system, supposing V is a set of words or items, we can transform the discrete index to a hidden space by using simple linear transformation as follows.

$$\mathbf{h}_i = \mathbf{W}\mathbf{x}_i + \mathbf{b} \tag{1}$$

where $\mathbf{W} \in \mathbb{R}^{d \times |V|}$ and $\mathbf{b} \in \mathbb{R}^d$ are the parameters of the embedding layer. Then, $\mathbf{x}_i \in \mathbb{R}^{|V|}$ is the one-hot representation of the word or item, and $\mathbf{h}_i$ is the hidden representation of x.

However, the trajectory grids are different from words or items. For words or items, the relations between them are totally hidden and thus hard to model. Therefore, we only use linear layers to embed them. However, as shown in Figure 1, the explicit spatial proximity exists among grids. Modeling the spatial proximity in an embedding layer can help the subsequential model to understand the relationship between the query trajectory and context trajectories, which will be empirically demonstrated in the experimental section.

To capture the spatial relationship among grids, we propose a method based on graph neural networks. Firstly, we construct a grid adjacent graph $\mathbf{A} \in \mathbb{R}^{|V| \times |V|}$ based on the grid adjacent relation where V is a set of grids. Specifically, we define two different graph construction methods which are shown in Figure 3.

In the first construction, we define four neighbors (denoted as 4N), i.e., left, right, up, and down, for the current grid. If the neighbor grid is not in the space grids, the neighbor grid will not be considered. Similar to 4N, in the second construction, we define eight neighbors (denoted as 8N), i.e., left, right, up, down, upper left, lower left, upper right and lower right, for the current grid. We will compare the performance of the two construction methods in the experiments.

After the grid graph is construction, we deliver graph A and the hidden representation of grids $\mathbf{H} \in \mathbb{R}^{|V| \times d}$ obtained by Equation 1, to the graph neural networks. Specifically, we first calculate $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}$, where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N$, $\mathbf{I}_N$ is an identity matrix, and $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$. Then we express the graph convolutional operation as follows.

$$\mathbf{H}_{k+1} = ReLU(\tilde{\mathbf{A}}\mathbf{H}_k\mathbf{W}_k^g) \tag{2}$$

where $\mathbf{H}_k$ presents the hidden representation of grids obtained by k-th graph convolution. $\mathbf{H}_0$ is the hidden representation of grids $\mathbf{H}$. $\mathbf{W}_k^g$ is the learnable weight matrix. $ReLU(x) = max(0, x)$ is the non-linear activation.

Since output of different convolution layer may contain different semantic information, fusing all the semantic information can reduce the risk of over-smoothing and improve the performance of the model. So we define a fusing function as follows.

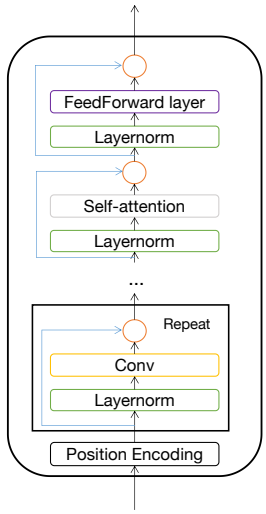


Fig. 4. The detail of QA-block. The QA-block contains five type of layers, i.e., position encoding layer, convolution layer, self-attention layer, feedforward layer and layernorm layer. The blue arrow means residual connection and the orange circle means sum operation.

$$\hat{\mathbf{H}} = Fuse(\mathbf{H}_1, \mathbf{H}_2, ..., \mathbf{H}_K) \tag{3}$$

where K is the maximum depth of graph convolution. $Fuse(\cdot)$ is an abstract function which can be replaced by other concrete differentiable functions. For example, *Sum* operation and *Concat* operation can be used here and Equation 3 can be converted to $\hat{\mathbf{H}} = \sum_{i=1}^{\mathbf{K}} \mathbf{H}_i$ and $\hat{\mathbf{H}} = Concat(\mathbf{H}_1, \mathbf{H}_2, ..., \mathbf{H}_K)$, separately. In our experiments, we use *Concat* operation and *MLP* with *ReLU* activation to fuse the different semantic information. For the input query $T_q =< q_1, q_2, ..., q_m >$ and context trajectory $T_c =< c_1, c_2, ..., c_n >$, we can obtain their corresponding spatial-aware embeddings $\hat{\mathbf{H}}_q \in \mathbb{R}^{m \times d}$ and $\hat{\mathbf{H}}_c \in \mathbb{R}^{n \times d}$ from the whole $\hat{\mathbf{H}}$.

4.2 Trajectory Embedding Module

The grids in a trajectory are not independent, among which sequential relation exists. It is important to capture this information when trajectory data is analyzed [16, 21, 44] and this module aims to model sequential information. We adopt two sequential methods, which are elaborated as follows.

The first method is the encoder-block, i.e., QA-block, used in QANet [46]. The QA-block is the predecessor of the transformer-block [32], which is the most popular sequential module in NLP. Specifically, Figure 4 shows the detail of QA-block. Due to the limited space, we do not present the calculation of QA-block, more details of which can be found in QANet [46].

Due to the difference between trajectories and natural language, i.e., short-term dependency versus long-term dependency, we use the recurrent neural network (RNN) as our the second method which is good at capturing recent information and extensively used in many sequential tasks such as time series prediction. Specifically, we use the bidirectional long-short term memory (BiLSTM) which is the variant of RNN and is able to model complex information than vanilla RNN. BiLSTM can not only model the information above but also the information blow which is beneficial for locating the span of the similar subtrajectories. We present the calculation of LSTM which contains three gates, i.e., input gate i^t, forget gate f^t and output gate o^t, and a cell state c^t.

$$
\begin{aligned}
\mathbf{i}^t &= \sigma(\mathbf{W}_i\mathbf{x}^t + \mathbf{U}_i\mathbf{h}^{t-1}) \\
\mathbf{f}^t &= \sigma(\mathbf{W}_f\mathbf{x}^t + \mathbf{U}_f\mathbf{h}^{t-1}) \\
\mathbf{o}^t &= \sigma(\mathbf{W}_o\mathbf{x}^t + \mathbf{U}_o\mathbf{h}^{t-1}) \\
\tilde{\mathbf{c}}^t &= tanh(\mathbf{W}_c\mathbf{x}^t + \mathbf{U}_c\mathbf{h}^{t-1}) \\
\mathbf{c}^t &= \mathbf{f}^t \circ \mathbf{c}^{t-1} + \mathbf{i}^t \circ \tilde{\mathbf{c}}^t \\
\mathbf{h}^t &= \mathbf{o}^t \circ tanh(\mathbf{c}^t)
\end{aligned}
\tag{4}
$$

where $\mathbf{W}_i \in \mathbb{R}^{d\times d}$, $\mathbf{U}_z \in \mathbb{R}^{d\times d}$, and $z \in \{i, f, o, c\}$ are parameters of LSTM. Further, $\circ$ represents element-wise multiplication. BiLSTM is a bidirectional LSTM, which models the original sequential information and the reversed sequential information. Specifically, we denote the vanilla LSTM as $\overrightarrow{LSTM}$, where over-right-arrow means that the sequence is left-to-right. Analogous to vanilla LSTM, we denote the reversed LSTM as $\overleftarrow{LSTM}$, where over-left-arrow means that the sequence is right-to-left. So the BiLSTM can be expressed as $BiLSTM = Concat(\overrightarrow{LSTM}, \overleftarrow{LSTM})$.

Through this module, the query embedding $\hat{\mathbf{H}}_q$ and the context embedding $\hat{\mathbf{H}}_c$ are converted to $\hat{\mathbf{H}}_{qs} \in \mathbb{R}^{m\times 2d}$ and $\hat{\mathbf{H}}_{cs} \in \mathbb{R}^{n\times 2d}$, separately. Inspired by the success of siamese networks [23], we share the trajectory embedding module between query and context. This operation can not only reduce the parameters of the model but also improve the performance which will be shown in the experimental part.

4.3 Query-Context Trajectory Fusion Module

To predict the span which is a portion of the context trajectory and is similar to the query trajectory based on a measurement $\Theta(\cdot,\cdot)$, it is necessary to fuse the query embedding with context. The paradigm of fusion is denoted by $QCFuse(\hat{\mathbf{H}}_{qs}, \hat{\mathbf{H}}_{cs})$, where $QCFuse$ is an abstract differentiable function. Here we present three concrete methods to fuse query with context.

The first query-context fusion method, namely naive fusion, is to directly combine the last hidden state of query trajectory embedding with context trajectory embedding. Specifically, we express it as follows.

$$
\hat{\mathbf{H}}_f = Concat(Repeat(\hat{\mathbf{H}}_{qs}[-1,:], n), \hat{\mathbf{H}}_{cs}) \tag{5}
$$

where $Repeat(\cdot,\cdot)$ represents a repetition function, the first parameter is the vector to be repeated, and the second parameter is the number of repetition. After the last hidden state of query trajectory embedding is repeated to have the same shape with context trajectory embedding, $\hat{\mathbf{H}}_f \in \mathbb{R}^{n\times 4d}$ is concatenated by the repeated vector and the context trajectory embedding.

However, this fusion method is too naive and intuitive since it only uses the last step of query trajectory embedding and ignores information from other steps. The ignorance of other steps will prevent generating more discriminative fusion feature $\hat{\mathbf{H}}_f$ and degrade the performance of the span prediction module. Thereby, to better leverage the query trajectory embedding, we propose two attention-based fusion methods. The first attention-based fusion method can be expressed as follows.

$$
\begin{aligned}
\hat{\mathbf{H}}_{ca} &= \hat{\mathbf{H}}_{cs}\mathbf{W}_{attn} \\
\hat{\mathbf{H}}_{qa} &= \hat{\mathbf{H}}_{qs}\mathbf{W}_{attn} \\
\mathrm{S} &= \hat{\mathbf{H}}_{ca} transpose(\hat{\mathbf{H}}_{qa}) \\
\mathbf{C}^{q}_{attn} &= Softmax(\mathrm{S})\hat{\mathbf{H}}_{qs} \\
\hat{\mathbf{H}}_{f} &= Concat(\hat{\mathbf{H}}_{cs}, \mathbf{C}^{q}_{attn}, \hat{\mathbf{H}}_{cs} \circ \mathbf{C}^{q}_{attn})
\end{aligned}
\tag{6}
$$

where $\mathbf{W}_{attn} \in \mathbb{R}^{d\times d}$ is the learnable parameters of attention. $\hat{\mathbf{H}}_{ca} \in \mathbb{R}^{n\times d}$ and $\hat{\mathbf{H}}_{qa} \in \mathbb{R}^{m\times d}$ are the intermediate values of attention weights. *transpose* is a function that returns transpose of the input matrix. $\mathrm{S} \in \mathbb{R}^{n\times m}$ is the similarity matrix. *Softmax* is a function, denoted by $exp(x_i)/\sum_j exp(x_j)$, which is used to normalize the similarity matrix and converts it to an attention matrix. $\mathbf{C}^{q}_{attn} \in \mathbb{R}^{n\times d}$ is the weighted sum of query trajectory embedding according to context trajectory embedding. Finally, *Concat* is used to fuse all parts to get the final fusion features $\hat{\mathbf{H}}_f \in \mathbb{R}^{n\times 6d}$, where $\circ$ is the element-wise multiplication. In the first attention-based fusion method, only one-directional attention, i.e., query-to-context, is considered. The second attention-based method consider bidirectional attention, i.e., query-to-context and context-to-query. To save the space, we omit the repetition parts in the follow expression.

$$
\begin{aligned}
\mathbf{Q}_{attn} &= Softmax(transpose(\mathrm{S}))\hat{\mathbf{H}}_{cs} \\
\mathbf{C}^{c}_{attn} &= Softmax(\mathrm{S})\mathbf{Q}_{attn} \\
\hat{\mathbf{H}}_{f} &= Concat(\hat{\mathbf{H}}_{cs}, \mathbf{C}^{q}_{attn}, \hat{\mathbf{H}}_{cs} \circ \mathbf{C}^{q}_{attn}, \hat{\mathbf{H}}_{cs} \circ \mathbf{C}^{c}_{attn})
\end{aligned}
\tag{7}
$$

where $\mathbf{Q}_{attn} \in \mathbb{R}^{m\times d}$ is the weighted sum of the context trajectory embedding according to query trajectory embedding. $\mathbf{C}^{c}_{attn} \in \mathbb{R}^{n\times d}$ is the weighted sum of $\mathbf{Q}_{attn}$. The operation of getting $\mathbf{C}^{c}_{attn}$ can achieve the deep fusion between query trajectory and context trajectory. Finally, *Concat* is used to get the final fusion features $\hat{\mathbf{H}}_f \in \mathbb{R}^{n\times 8d}$ [2].

4.4 Span Prediction Module

Given fusion features of the query trajectory and context trajectory, the span prediction module aims to predict the probability of each point to be the start and the end of the subtrajectory.

$$
\begin{aligned}
\mathbf{H}_p &= ReLU(ReLU(\hat{\mathbf{H}}_f\mathbf{W}_{p1} + \mathbf{b}_{p1})\mathbf{W}_{p2} + \mathbf{b}_{p2}) \\
\mathbf{p}_s &= Softmax(\mathbf{H}_p\mathbf{W}_s) \\
\mathbf{p}_e &= Softmax(\mathbf{H}_p\mathbf{W}_e)
\end{aligned}
\tag{8}
$$

where $\mathbf{W}_{p1} \in \mathbb{R}^{4d\times d}, \mathbf{W}_{p2} \in \mathbb{R}^{d\times d}, \mathbf{b}_{p1}, \mathbf{b}_{p2} \in \mathbb{R}^{d}, \mathbf{W}_s, \mathbf{W}_e \in \mathbb{R}^{d}$ are learnable variable[3]. $\mathbf{H}_p \in \mathbb{R}^{n\times d}$ is the intermediate variables that is the output of two stacked linear layers with *ReLU* activation. $\mathbf{p}_s \in \mathbb{R}^{n}$ and $\mathbf{p}_e \in \mathbb{R}^{n}$ are the probability of each point to be the start and the end of the subtrajectory respectively.

[2]Without special instruction, the attention-based fusion method denotes the bidirectional attention-based fusion method.
[3]We express the dimension when the naive fusion method is used. The dimension of $\mathbf{W}_{p1}$ in the attention-based fusion method are $\mathbb{R}^{6d\times d}$ and $\mathbb{R}^{8d\times d}$ respectively for the two attention-based fusion methods.

5 MODEL TRAINING AND INFERENCE

In this section, the loss function used to train the model will be shown. And we also present how to infer the subtrajectory after the start probability and the end probability are obtained.

5.1 Model Training

The objective function is defined as the negative sum of the log probabilities of the predicted start and end probability, averaged over all the training examples.

$$L(\theta) = -\frac{1}{|D|}\sum_{i}^{|D|}[log(\mathbf{p}_s[s_i]) + log(\mathbf{p}_e[e_i])] \tag{9}$$

where $\mathbf{p}_s$ and $\mathbf{p}_e$ are predicted start and end probability, respectively. s_i and e_i are the ground truth starting and ending positions of the i-th example in dataset D. θ presents all the trainable parameters. The ground truth is generated by ExactS algorithm [38]. We will show the detail of this algorithm in the experimental section.

5.2 Inference

After start probability and end probability is predicted by the span prediction module, the subtrajectory can be obtained. The subtrajectory can be chosen by a span (i, j), where $\mathbf{p}_s[i]\mathbf{p}_e[j]$ is maximized and $i \leq j$, i.e., $T_c[i, j]$. In the implementation of the inference, we convert the shape of $\mathbf{p}_s$ and $\mathbf{p}_e$ to $(n, 1)$ and $(1, n)$, denoted by $\mathbf{P}_s$ and $\mathbf{P}_e$ respectively. Then the inference process can be expressed as follows.

$$\begin{aligned} \mathbf{M} &= Triu(\mathbf{P}_s\mathbf{P}_e) \\ i^*, j^* &= MatrixArgmax_{i,j}\ (\mathbf{M}) \end{aligned} \tag{10}$$

where *Triu* returns the upper triangular matrix of the input matrix. $MatrixArgmax(\mathbf{M})$ is a function that returns the row i^* and the column index j^*, where $\mathbf{M}[i^*, j^*]$ is the maximum value in M. There is a trick to speedup the process of inference. We express the inference process with speedup trick in Algorithm 1. The algorithm starts with the calculation of i^t and j^t (line 1), which takes $\mathbf{p_s}$ and $\mathbf{p_e}$ as inputs. According to the definition of the *Argmax*, we can know that $\mathbf{p}_s[i^t]\mathbf{p}_e[j^t]$ is the maximum value among $\mathbf{M}$. If $i^t \leq j^t$, which guarantees that $T_c[i^t, j^t]$ is a subtrajectory, $T_c[i^t, j^t]$ is the most similar subtrajectory to query trajectory (line 2). If $i^t > j^t$, which means $T_c[i^t, j^t]$ is not a subtrajectory, we should calculate i^* and j^* according to Equation 10 (line 3). It is obvious that a well-trained model can satisfy the condition $i^t \leq j^t$ in most cases, which make our algorithm only need to calculate *Argmax* rather than *MatrixArgmax* in these cases. Moreover, the time complexity of *Argmax* calculation on $\mathbf{p}_s$ or $\mathbf{p}_e$ is $O(n)$ and the time complexity of *MatrixArgmax* calculation on $\mathbf{M}$ is $O(n^2)$. Therefore, Algorithm 1 is more efficient than the direct calculation of Equation 10.

6 EXPERIMENTS

We present the experimental set-up and the experimental results in this section.

6.1 Experimental Setup

6.1.1 Datasets. Our experiments are conducted on two real-world trajectory datasets. The first dataset is Porto which is collected from the city of Porto, Portugal. This dataset consists around 1.7 million taxi trajectories over 18 months with a sampling interval of 15 seconds and a mean length around 60. The second dataset, denoted by Harbin, involves around 1.2 million taxi trajectories

Algorithm 1 Inference with speedup trick

Input: Predicted start probability $\mathbf{p}_s$ and end probability $\mathbf{p}_e$;
Output: The start and end tuple (i^*, j^*) of predicted subtrajectory;

1: Calculating i^t=$Argmax(\mathbf{p}_s)$ and j^t=$Argmax(\mathbf{p}_s)$ // where $Argmax(v)$ returns a index of maximum value in vector $\mathbf{v}$;
2: **if** $i^t \leq j^t$: $i^* = i^t$; $j^* = j^t$; // if $i^t \leq j^t$, $\mathbf{p}_s[i^t], \mathbf{p}_e[j^t] = MatrixArgmax_{i,j}(\mathbf{M})$, where $M = Triu(\mathbf{P}_s\mathbf{P}_e)$
3: **else** $\mathbf{M} = Triu(\mathbf{P}_s\mathbf{P}_e)$; $i^*, j^* = MatrixArgmax_{i,j}(\mathbf{M})$;
4: **return** (i^*, j^*)

collected from 13000 taxis over 8 months in Harbin, China with non-uniform sampling rates and an average length of around 120.

6.1.2 Baselines. We compare our proposed framework with similar subtrajectory search methods and reading comprehension methods. We choose five similar subtrajectory search methods from [38] and two reading comprehension models from [46]. (1) **SizeS**: ExactS explores all possible subtrajectories, many of which might be quite dissimilar from the query trajectory and could be ignored. This algorithm restricts the search space of similar subtrajectories only to those trajectories with similar sizes, i.e., $[m - \xi, m + \xi]$, for better efficiency. (2) **POS**: This algorithm is a greedy one and based on splitting strategy. When it scans a point p_i, it computes the similarities between the subtrajectories that would be formed if it splits at p_i, i.e., $T_c[h, i]$, and the query trajectory T_q, where p_h is the point following the one, at which the last split was done if any and p_h is the first point p_1 otherwise. (3) **POS-D**: POS-D is the delay version of POS. Whenever a prefix is found to be more similar to the query trajectory than the current best subtrajectory, POS-D continues to scan n more points and splits at one of these $n + 1$ points. (4) **RLS**: RLS is a reinforcement learning based search algorithm, which formulates the process of splitting a context trajectory as a MDP. Then the Deep-Q-Network (DQN) is used to model the MDP. In DQN, we define two actions, i.e $a = 1$ and $a = 0$, which means performing a split operation at the point that is being scanned and moving on to scan the next point, respectively. (5) **RLS-Skip**: RLS-Skip introduce k more actions than RLS, namely skipping 1 point, skipping 2 points, ..., skipping k points which means skipping the next k point. This skip operation will reduce the search space. (6) **QANet**: QANet is the most common used reading comprehension model which uses the QA-block as the sequential encoder after word embedding. It stacks 7 QA-block layers, called model encoder layer, after fusing query and context. Besides, it shares weights between each of three repetitions of the model encoder. All the output of the three repetitions are used to predict the span. (7) **G-QANet**: G-QANet is the QANet based on spatial-aware embedding.

6.1.3 Ground-truth generation. To generate the ground truth of the start and the end position of the subtrajectories, we use the $ExactS$[4] algorithm proposed by [38]. The ExactS algorithm enumerates all the possible subtrajectories $T[i, j](1 \leq i \leq j \leq n)$ of the context trajectory T, computes the similarity between $T[i, j]$ and T_q, i.e., $\Theta(T[i, j], T_q)$ and then returns the one with the greatest similarity. To avoid the noise pairs, i.e., the query and context trajectory are totally different, we filter out the pairs when the returned subtrajectory is shorter than a threshold ϵ. In our experiments, the threshold ϵ is set to 5. For the both datasets, we randomly choose 24500/2000/10500 query and context trajectories pairs as training/validation/testing set.

[4]https://github.com/zhengwang125/SimSub. This repository which is opened by the original authors, not only contain the ExactS algorithm but also RLS, RLS-Skip, SizeS, POS, POS-D used as baselines in this paper. The Porto and Harbin dataset can be obtained in this repository.

6.1.4 Parameter setting. For training t2vec model, we follow the other similar subtrajectory search paper [38] by excluding those trajectories that are short and setting the grid size at $100m \times 100m$ for Porto and Harbin. Other hyperparameters are the same with the original paper. After the trained t2vec is obtained, we use the ground-truth generation which has shown before to generate training/validation/testing set. For all the baselines, we use the same parameters with the original papers [38, 46]. Specifically, for SizeS, we use the setting $\xi = 5$. For POS-D, we use the setting $n = 5$. For RLS and RLS-Skip, we use a 2 layer feedforward neural network. In the first layer, 20 neurons and *ReLU* activation are used. In the second layer, we use *sigmoid* activation with $2+k$ neurons. The k in RLS-Skip is set at 3. For all of our models, QANet and G-QANet, we set the hidden size $d = 128$. All the trainable models use Adam optimizer to learn the model and the learning rate r is set at 0.001. The batch size is fixed at 32 both for training and testing phases. The maximal training epoch is set to 50, and we use early stop in our models to prevent overfitting. Specifically, when the validation loss does not degrade in continuous e epochs ,i.e., 5 epochs, we stop training and use the model which has the minimum validation loss as the final model.

6.1.5 Evaluation metric. We use three metrics [38] to evaluate the effectiveness of all the models. (1) Approximate Ratio (AR): It is defined as the ratio between the dissimilarity of the solution w.r.t. a query trajectory, which is returned by baselines, and that of the solution returned by ExactS algorithm. (2) Mean Rank (MR): We sort all the subtrajectories of a context trajectory in ascending order of their dissimilarities w.r.t. a query trajectory. Mean Rank is defined as the rank of the solution returned by a model. (3) Relative Rank (RR): Relative Rank is a normalized version of Mean Rank by total number subtrajectories of a context trajectory. A smaller value of metrics indicate a better model.

6.1.6 Evaluation platform. The implementation of our proposed framework is based on Pytorch 1.5.0 and Torch_geometric 1.6.0. All the models are implemented on a server with 48-cores Intel(R) Xeon(R) CPU E5-2650 v4 @ 2.20GHz 256GB RAM and one TITAN Xp GPU.

6.1.7 Concrete model derived from the proposed framework. Our proposed method is a framework rather than a concrete model. Here, we present some variants derived from the proposed framework. We will discuss their performance in the following experiments. In Table 2, EMB, TEM, QCTFM and SPM are the abbreviations of spatial-aware grid EMBedding module, Trajectory Embedding Module, Query-Context Trajectory Fusion Module and Span Prediction Module respectively. QANet and G-QANet also can be expressed by our framework, as shown in table 3.

Table 2. Models derived from the proposed framework.

Models	EMB	TEM	QCTFM	SPM
LBAF	Linear	BiLSTM	Attention-based Fusion	Feedforward with ReLU
GBNF	GNN	BiLSTM	Naive Fusion	Feedforward with ReLU
GQAF	GNN	QA-block	Attention-based Fusion	Feedforward with ReLU
GBAF	GNN	BiLSTM	Attention-based Fusion	Feedforward with ReLU

6.2 Experimental Results

6.2.1 Overall performance. We compare our proposed model GBAF with the baseline models on two datasets. In this comparison, GBAF and G-QANet both use two GNN layers with 8N graph construction in the embedding module. The results are shown in Table 4. Through this table, we

Table 3. QANet and G-QANet expressed by our framework.

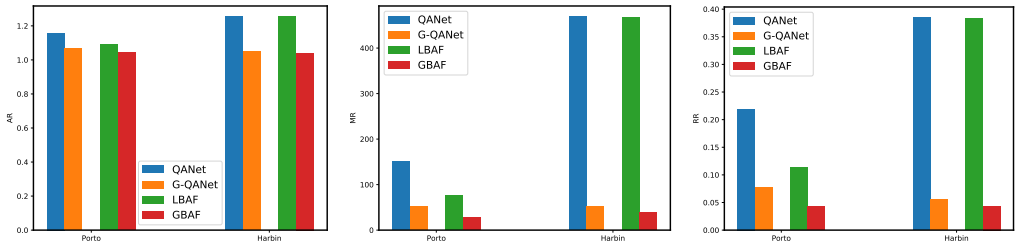
Models	EMB	TEM	QCTFM	SPM
QANet	Linear	QA-block	Attention-based Fusion	7 layers QA-block + Linear layer
G-QANet	GNN	QA-block	Attention-based Fusion	7 layers QA-block + Linear layer

Table 4. Overall performance on three datasets. The best results are marked in **bold** and the second results are underlined.

Methods	Porto			Harbin		
	AR	MR	RR	AR	MR	RR
SizeS	1.1241	87.38	0.1651	1.1211	116.71	0.1701
POS-D	1.1360	80.45	0.1290	1.0946	73.11	0.0973
POS	1.1697	106.63	0.1653	1.1600	164.72	0.1916
RLS	1.0617	35.85	0.0551	1.0423	40.96	0.0452
RLS-Skip	1.0648	36.40	0.0559	1.0471	45.58	0.0503
G-QANet	1.0689	52.00	0.0770	1.0515	52.61	0.0551
GBAF	**1.0423**	**27.61**	**0.0426**	**1.0409**	**39.48**	**0.0438**

can see that the learning methods, i.e., RLS, RLS-Skip, G-QANet, and GBAF, consistently get better performance than other methods, i.e., SizeS, POS-D, and POS. Specifically, GBAF outperforms SizeS in terms of MR by 68% in the Porto dataset. RLS outperforms POS-D in terms of RR by 44% in the Harbin dataset. POS-D can get better performance than POS which shows the delay strategy in POS-D is effective. RLS consistently outperforms than RLS-Skip because RLS-Skip focuses on efficiency and skips some points which should be scanned. Through this table, we can find that the performance of the models have a large gap between the two datasets, i.e, the performance of the models in Harbin is worse than that in Porto, which is because the mean length of the Porto dataset is less than that of the Harbin dataset, and it is harder to find the most similar subtrajectories to query trajectory in the longer trajectories. The comparison of efficiency will be shown in later experiments. Moreover, we also observe that is GBAF can consistently get smaller values on three metrics than all the baselines on all datasets which shows the effectiveness of our proposed model.

6.2.2 Effectiveness of spatial-aware grid embedding. To proof the effectiveness of the spatial-aware grid embedding module, we compare GBAF, LBAF, QANet and G-QANet models. The detail of these model can be seen in Tables 2 and 3. The results are shown in Figure 5, which illustrate that G-QANet and GBAF, equipped with spatial-aware grid embedding module, have lower metrics value, i.e., AR, MR, and RR, than QANet and LBAF. Specifically, in Porto dataset, GBAF outperforms LBAF in terms of MR by 63% and G-QANet outperforms QANet in terms of RR by 64%. In Harbin dataset, GBAF outperforms LBAF in terms of RR by 59% and G-QANet outperforms QANet in terms of MR by 64%. This observation shows the effectiveness of the proposed spatial-aware grid embedding module. Without the spatial-aware grid embedding module, QANet and LBAF perform worse than most of the baselines. Moreover, G-QANet is a more complex model with advanced attention-based QA-block than GBAF. There is an interesting finding that the performance of GBAF is better than G-QANet which shows that state-of-the-art model in NLP may not perform well by directly using in trajectory data.



(a) Approximate Ratio (b) Mean Rank (c) Relative Rank

Fig. 5. Performance of QANet, G-QANet, GBAF and LBAF on two datasets about three metrics.

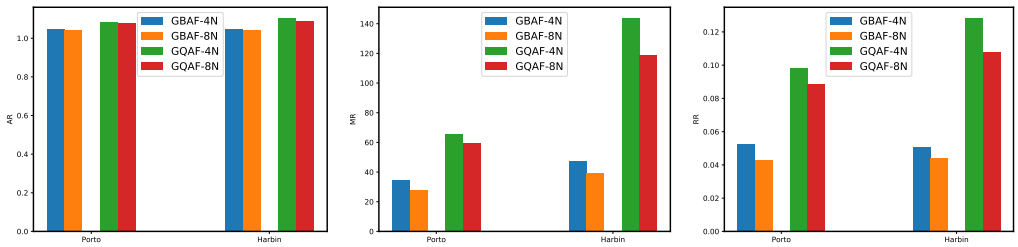


Fig. 6. The performance of the two proposed graph construction. The suffix 4N and 8N means the graph construction is 4N or 8N.

6.2.3 4N v.s. 8N graph construction. Next, we compare the performance of the proposed two graph construction methods, i.e., 4N and 8N. Figure 6 shows the results. In this Figure, we can see that the 8N graph construction method is consistently better than 4N. This is because the eight neighbors which is around the central grid can describe spatial relations between grids more accurately than four neighbors. The performance have a clear gap between the two construction methods which shows that the graph construction have a big impact on the performance. Due to the limited space, we will explore more grid graph construction methods in the future works.

6.2.4 The performance of GNN variants. In this paper, we mainly focus on the efficiency. However, the vanilla GCN [12] is very time-consuming due to the non-linear mapping. Therefore, we use simpler GCN-related models, i.e., LightGCN [10], and SGC [39], to get similar performance and save the time cost. Besides, we use a more complex GNN, i.e., GAT [33], to improve the performance. As shown in Figure 7, in the Porto dataset, SGC [39] is better than GAT which uses an attention strategy to aggregate the information of neighbors and is more complex than SGC. This shows that the complex model is not always better than the simpler models. In Harbin dataset, vanilla GCN performs the best among the methods. The results are totally different from those in the Porto dataset which shows that in a dataset we should conduct experiments to choose the most appropriate graph neural networks.

6.2.5 Siamese v.s. Pseudo-siamese trajectory embedding module. Siamese network [23] is the most common architecture in NLP which is implemented by shared weights of neural networks. In trajectory embedding module, the embedding of query and context trajectory can be encoded by two separated sequential models, namely Pseudo-siamese architecture, or by a shared weights sequential

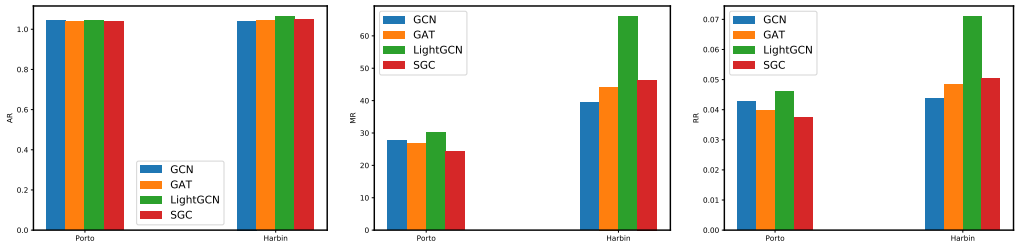


Fig. 7. The performance of GNN variants.

Table 5. The performance of Siamese architecture and Pseudo-siamese architecture about trajectory embedding module.

Methods	Porto			Harbin		
	AR	MR	RR	AR	MR	RR
GBAF-Siamese	1.0423	27.61	0.0426	1.0409	39.48	0.0438
GBAF-Pseudo-siamese	1.0547	39.20	0.0599	1.0507	50.83	0.0550
G-QANet-Siamese	1.0623	45.71	0.0673	1.0520	51.95	0.0547
G-QANet-Pseudo-siamese	1.0689	52.00	0.0770	1.0515	52.61	0.0551

model, namely Siamese. In this part, we investigate the performance of the two architectures. The comparison of the results are shown in Table 5. Through this table, we can see that GBAF-Siamese and G-QANet-Siamese perform worse than GBAF-Pseudo-siamese and G-QANet-Pseudo-Siamese, respectively, which shows that the Siamese architecture is better than Pseudo-siamese architecture in this task. Comparing the performance of G-QANet-Siamese and G-QANet-Pseudo-siamese models in Porto dataset, we can find that there is a large improvement by using Siamese architecture. This is due to the fact that the Siamese network can map the query and context trajectories to the same semantic space which is helpful for capturing the common semantic by the fusion module. The effectiveness of Siamese architecture recommends us to design a shared weights architecture in trajectory embedding module to solve the SimSub problem.

6.2.6 Naive v.s. attention-based fusion method. In the fusion module, we propose three concrete fusion methods, i.e., one naive fusion method and two attention-based fusion methods. We compare the performance of three models, i.e., GBNF-Naive, GBAF-OD, and GBAF-BiD, where OD is the One-Directional fusion method and BiD is the BiDirectional fusion method, respectively. As shown in Figure 8, we can see the Naive fusion method is the worst among all the three fusion methods. The one-directional attention-based method achieves a large improvement compared to the Naive fusion method. The performance of the two attention-based fusion methods are very close, but the bidirectional attention-based fusion method is a little better than the other method which shows that only query-to-context attention is not enough to capture the fusion information.

6.2.7 The effect of query trajectory length. We randomly select 10000 query and context trajectory pairs and partition the pairs into four groups, i.e., $G1$, $G2$, $G3$, and $G4$, according to the length of the query trajectory, where the range of the four group are $[10, 30)$, $[30, 50)$, $[50, 70)$, $[70, 90)$. For each group, we report performance of our models and some baselines, i.e., POS, POS-D, and SizeS. The results is shown in Figure 9. Through this Figure, we can see that POS, POS-D and GBAF can

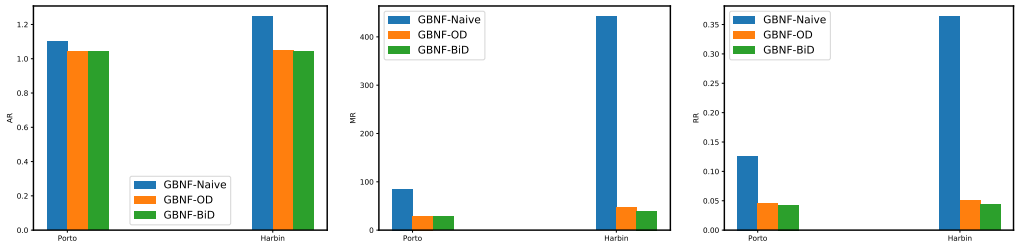


Fig. 8. Performance of comparison among three fusion methods.

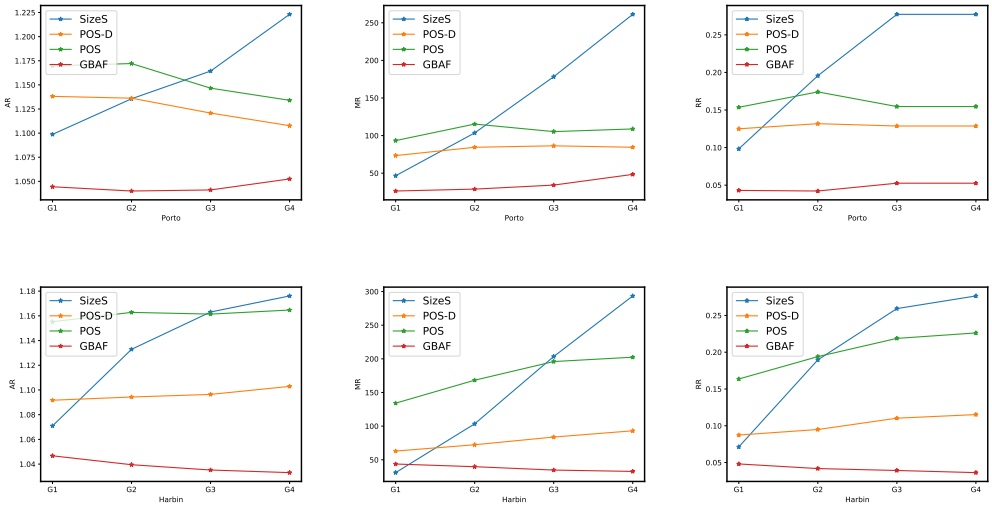


Fig. 9. The effect of query trajectory length on two datasets in terms of all metrics.

get consistent performance no matter the change of the query trajectory length. However, the SizeS is strongly affected by the length of the query trajectory. Specifically, SizeS can get pretty low AR, MR, and RR values in shorter query trajectory length. But with the growth of the query trajectory length, the performance of SizeS drop sharply. In most of the cases, our GBAF performs better than other models except for MR of $G1$ in Harbin dataset, in which SizeS gets the best performance.

6.2.8 The effect of ϵ. The ϵ is used to filter the noise query and context trajectory pairs which will make the model hardly to train. In this set of experiments, we fix the ϵ at five in the training phase and change ϵ to generate a testing dataset which is more in line with real scenario, where the query and context trajectories may be different. Figure 10 presents the results where ϵ varies from 1 to 5. The results show that the learnable methods perform consistently better than non-learning methods. Moreover, with the increase of ϵ, the learnable methods have not large changes which shows that the learnable methods are stable regardless of the changes of ϵ. In this figure, we can see that our GBAF always outperforms the others which shows the effectiveness of the proposed GBAF model.

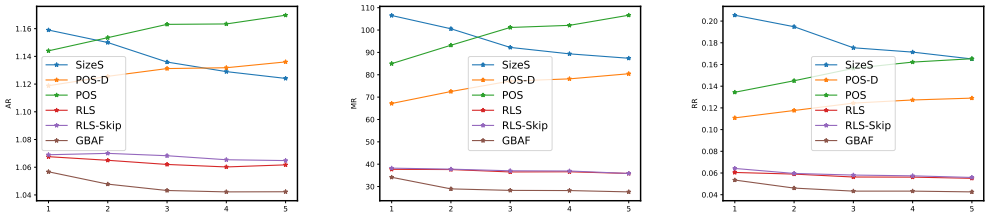


Fig. 10. The effect of ϵ on Porto dataset.

Table 6. Training time of learnable models (hours).

	RLS	RLS-Skip	G-QANet	GBAF
Porto	47.85	41.84	0.43	0.22
Harbin	90.32	70.81	0.88	0.27

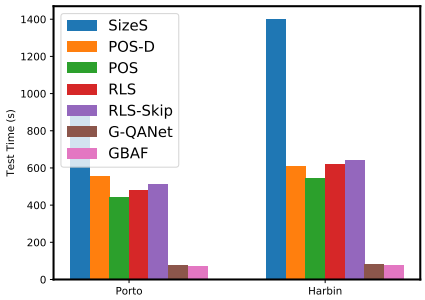


Fig. 11. Inference time of all the methods on two datasets.

6.2.9 Training time. To show the efficiency of our proposed framework, we compare the training time of GBAF with the other learnable models, i.e., RLS, RLS-Skip, and QANet. The results are shown in Table 6. We can see that the reinforcement learning methods, i.e., RLS, and RLS-Skip, cost too much time to be trained which are infeasible in real applications. Even the RLS-Skip which is an efficient version of RLS also cost almost two and three days in Porto and Harbin datasets, respectively. Compared to these methods, the models derived from our framework is faster in training phase. Specifically, GBAF obtains 217x speedup over RLS method and 190x speedup over RLS-Skip method in Porto dataset. Moreover, GBAF is more efficient than G-QANet which shows that the existing reading comprehension models may not efficient enough in SimSub problem.

6.2.10 Inference time. Inference time is an important evaluation metric about efficiency. In this part, we compare the inference time of all the models. The results is shown in Figure 11. Through this figure, we can see that SizeS is the most time-consuming method which is almost two times slower than other baselines, i.e., RLS, RLS-Skip, POS-D, and POS. The inference time of POS-D is more than that of POS because of the delay strategy. There is an interesting finding that RLS is

Table 7. Case study of top-k subtrajectories returned by algorithms.

$topk$	POS		POS-D		RLS		RLS-Skip		SizeS		GBAF	
	span	rank	span	rank	span	rank	span	rank	span	rank	span	rank
$top1$	(14, 26)	41	(26,26)	25	(0,26)	20	(0,26)	20	(0,26)	20	(5,26)	1
$top2$	-	-	-	-	-	-	-	-	(5,31)	62	(4,26)	5
$top3$	-	-	-	-	-	-	-	-	(8,44)	66	(5,27)	18
$top4$	-	-	-	-	-	-	-	-	(4,31)	75	(5,25)	2
$top4$	-	-	-	-	-	-	-	-	(8,43)	85	(13,26)	21

faster than RLS-Skip in the inference phase which is opposite in the training phase. This is because in the training phase, the time of gradient backpropagate makes up the bulk of the total training time, and jumping k points will reduce the calculation of gradient. Therefore, the training time will be reduced by replacing RLS with RLS-Skip. However, in the inference phase, the time of choosing states dominiates the testing time, and RLS-Skip contains k states more than RLS, which will cause more testing time. Moreover, GBAF and G-QANet are the most efficient methods which show large improvement compared to other methods. Specifically, GBAF gets around 8x speedup over the RLS in the Harbin dataset. G-QANet obtains 5x speedup over POS which is the most efficient non-learning method. The performance of GBAF and G-QANet shows the high efficiency of the proposed framework.

6.2.11 Case study. Except for the ExactS [38], the other algorithms including all the baselines and our models are approximate methods. In some cases, we need to get more candidate similar subtrajectories, i.e., k subtrajectories rather than only one subtrajectory, for subsequential tasks. This goal cannot be directly achieved by the splitting algorithm, i.e., POS, POS-D, RLS, and RLS-Skip. However, thanks to the span prediction module, the proposed framework can not only return the most similar subtrajectory of context trajectory for the query trajectory, but also the top-k similar subtrajectories which can be seen in Figure 12. We randomly choose a query and context trajectory pair from the Porto dataset, then show the subtrajectories predicted by all the models and its corresponding true ranks returned by ExactS which is shown in Table 7. In this table, we can see that SizeS and our GBAF can return top-k similar subtrajectories but POS, POS-D, RLS and RLS-Skip only return one similar subtrajectory. Besides, the rank of returned top-k subtrajectories by GBAF are smaller than those by SizeS, which means that these subtrajectories are more similar to the query trajectory than those returned by SizeS. Moreover, the top-k in SizeS maybe not stable. For example, when the length of query trajectory and context trajectory are 32 and 27, respectively, and ξ is set to 5, SizeS only compares one subtrajectory, i.e., [0,26], to query trajectory which means that the k must be 1. In addition, the retured subtrajectory by POS-D is only one point, i.e., (26,26), which maybe returned by any splitting-based algorithms and it is not easy to constrain the returned length of trajectories by the splitting-based algorithms.

In real-world applications, the similar subtrajectories may have some restrictions such as length. Only one point involved in the returned subtrajectory is nonsense. Similar to returning the top-k subtrajectories, our framework can also constrain the length of returned subtrajectories by operating the predicted start and end probability. The right of Figure 12 shows the details. In this figure, the diagonal elements of the start-end matrix are set to 0. The left non-shadow part guarantees that the index of the end minus the index of the start is larger than 1 which means that the length of returned subtrajectory is larger than 1. Analogous to the simple case, if we set the entries to 0 along

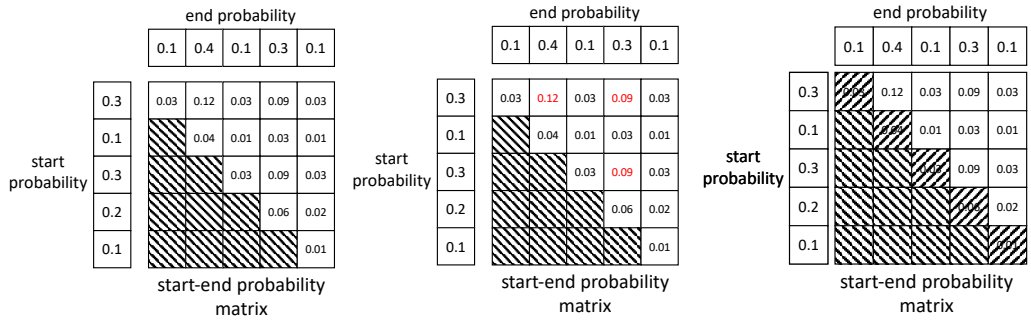


Fig. 12. The example of case study. The left subfigure shows the construction of start-end matrix from start and end probability. The shadow parts in this subfigure is set to zeros which means the end of the subtrajectory should be larger than the start. The middle subfigure shows that our proposed framework can return more than one subtrajectory in inference phase. The red number in this subfigure is top-3 similar subtrajectories to the query trajectory. And how to constrain the length of returned subtrajectory is shown in the right subfigure. In this subfigure, we set the diagonal elements of the start-end matrix to zeros which means that the length of returned subtrajectory must be larger than one.

the diagonal, we can constrain the returned subtrajectory to any length, which can not be achieved by the existing methods.

Summary: The take-away message of our empirical study can be summarized as follows:

(1) The models equiped with a spatial-aware grid embedding module have larger improvements than the models without this module.
(2) The choices of the neighbors and the types of used GNN in the spatial-aware grid embedding module have a large affect on the performance.
(3) BiLSTM can extract more effective trajectory information than QA-block, which shows the difference between trajectories and natural languages.
(4) The Siamese network architecture can use fewer learnable parameters to achieve better performance.
(5) The bidirectional attention-based fusion method, i.e, query-to-context and context-to-query, is better than other fusion methods.
(6) Compared with all the baselines, the proposed GBAF achieves the best efficiency and effectiveness.
(7) Although RLS-Skip is faster than RLS in the training phase, RLS is more efficient than RLS-Skip in the inference phase. Moreover, the models derived from the proposed 3SGNN framework are more effective than the traditional methods and reinforcement-based methods.

7 CONCLUSION

In this paper, we aim to tackle the similar subtrajectory search problem. We find that this problem can be formulated as a reading comprehension problem when the grid-based metric is used. Therefore, we adopt t2vec, a grid-based deep trajectory similarity measurement, as the metric. In order to achieve high effectiveness and efficiency, we addressed a few challenges by proposing a 3SGNN framework, which is a general framwork that can combine different models. Extensive empirical studies on real world datasets confirms the effectiveness and efficiency of our proposed framework compared with the state-of-the-art methods.

ACKNOWLEDGEMENT

This work is supported by NSFC (No. 61972069, 61836007, 61832017).

REFERENCES

[1] P. Agarwal, K. Fox, K. Munagala, A. Nath, J. Pan, and E. Taylor. 2018. Subtrajectory Clustering: Models and Algorithms. *SIGMOD/PODS* (2018).

[2] H. Alt and M. Godau. 1995. Computing the Fréchet distance between two polygonal curves. *Int. J. Comput. Geom. Appl.* 5 (1995), 75–91.

[3] P. Baldi, S. Brunak, P. Frasconi, G. Soda, and G. Pollastri. 1999. Exploiting the past and the future in protein secondary structure prediction. *Bioinformatics* 15 11 (1999), 937–46.

[4] K. Buchin, M. Buchin, J. Gudmundsson, M. Löffler, and J. Luo. 2011. Detecting Commuting Patterns by Clustering Subtrajectories. *Int. J. Comput. Geom. Appl.* 21 (2011), 253–282.

[5] L. Chen and R. Ng. 2004. On The Marriage of Lp-norms and Edit Distance. In *VLDB*.

[6] L. Chen, M. Özsu, and V. Oria. 2005. Robust and fast similarity search for moving object trajectories. In *SIGMOD*.

[7] Y. Cui, L. Deng, Y. Zhao, B. Yao, V. W. Zheng, and K. Zheng. 2019. Hidden POI Ranking with Spatial Crowdsourcing. In *SIGKDD*. 814–824.

[8] J. Devlin, M. Chang, K. Lee, and K. Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *NAACL-HLT*.

[9] R. H. Güting and M. Schneider. 2005. Realm-based spatial data types: The ROSE algebra. *VLDB* 4 (2005), 243–286.

[10] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang. 2020. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. *SIGIR* (2020).

[11] S. Kanda, K. Takeuchi, K. Fujii, and Y. Tabei. 2020. Succinct Trit-array Trie for Scalable Trajectory Similarity Search. *SIGSPATIAL* (2020).

[12] T. Kipf and M. Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. *ArXiv* abs/1609.02907 (2017).

[13] S. Koide, C. Xiao, and Y. Ishikawa. 2020. Fast subtrajectory similarity search in road networks under weighted edit distance constraints. *PVLDB* 13 (2020), 2188 – 2201.

[14] J. Lee, J. Han, and K. Whang. 2007. Trajectory clustering: a partition-and-group framework. In *SIGMOD '07*.

[15] L. Li, K. Zheng, S. Wang, W. Hua, and X. Zhou. 2018. Go slow to go fast: minimal on-road time route scheduling with parking facilities using historical trajectory. *The VLDB Journal* 27 (2018), 321–345.

[16] X. Li, K. Zhao, G. Cong, C. S. Jensen, and W. Wei. 2018. Deep Representation Learning for Trajectory Similarity Computation. *ICDE* (2018), 617–628.

[17] X. Li, Y. Zhao, J. Guo, and K. Zheng. 2020. Group Task Assignment with Social Impact-based Preference in Spatial Crowdsourcing. In *DASFAA*. 677–693.

[18] X. Li, Y. Zhao, K. Zheng, and X. Zhou. 2020. Consensus-Based Group Task Assignment with Social Impact in Spatial Crowdsourcing. *Data Science and Engineering* 5, 4 (2020), 375–390.

[19] G. Liu, Y. Liu, K. Zheng, An Liu, Z. Li, Y. Wang, and X. Zhou. 2018. MCS-GPM: Multi-Constrained Simulation Based Graph Pattern Matching in Contextual Social Graphs. *IEEE Transactions on Knowledge and Data Engineering* 30 (2018), 1050–1064.

[20] G. Liu, Y. Wang, and M. Orgun. 2013. Finding the Optimal Social Trust Path for the Selection of Trustworthy Service Providers in Complex Social Networks. *IEEE Transactions on Services Computing* 6 (2013), 152–167.

[21] Y. Liu, K. Zhao, G. Cong, and Z. Bao. 2020. Online Anomalous Trajectory Detection with Deep Generative Sequence Modeling. *ICDE* (2020), 949–960.

[22] A. Maheshwari, J. Sack, and C. Scheffer. 2015. Approximating the Integral Fréchet Distance. *arXiv* (2015).

[23] W. Pei, D. Tax, and L. V. D. Maaten. 2016. Modeling Time Series Similarity with Siamese Recurrent Networks. *ArXiv* (2016).

[24] T. Rakthanmanon, Bilson J. L. Campana, A. Mueen, Gustavo E. A. P. A. Batista, M. Westover, Q. Zhu, J. Zakaria, and E. J. Keogh. 2012. Searching and Mining Trillions of Time Series Subsequences under Dynamic Time Warping. *KDD* 2012 (2012), 262 – 270.

[25] T. Rakthanmanon, B. J. L. Campana, A. Mueen, G. E. A. P. A. Batista, M. Westover, Q. Zhu, J. Zakaria, and E. J. Keogh. 2013. Addressing Big Data Time Series: Mining Trillions of Time Series Subsequences Under Dynamic Time Warping. *TKDD* 7 3 (2013).

[26] S. Ranu, P Deepak, A. Telang, P. Deshpande, and S. Raghavan. 2015. Indexing and matching trajectories under inconsistent sampling rates. *ICDE* (2015), 999–1010.

[27] Y. Sakurai, C. Faloutsos, and M. Yamamuro. 2007. Stream Monitoring under the Time Warping Distance. *ICDE* (2007), 1046–1055.

[28] M. Schuster and K. Paliwal. 1997. Bidirectional recurrent neural networks. *IEEE Trans. Signal Process.* 45 (1997), 2673–2681.
[29] M. Seo, A. Kembhavi, A. Farhadi, and H. Hajishirzi. 2017. Bidirectional Attention Flow for Machine Comprehension. *ArXiv* abs/1611.01603 (2017).
[30] H. Su, S. Liu, B. Zheng, X. Zhou, and K. Zheng. 2019. A survey of trajectory distance measures and performance evaluation. *The VLDB Journal* 29 (2019), 3–32.
[31] H. Su, K. Zheng, J. Huang, H. Wang, and X. Zhou. 2014. Calibrating trajectory data for spatio-temporal similarity analysis. *The VLDB Journal* 24 (2014), 93–116.
[32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. 2017. Attention is All you Need. In *NIPS*.
[33] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. 2018. Graph Attention Networks. *ArXiv* abs/1710.10903 (2018).
[34] O. Vinyals, M. Fortunato, and N. Jaitly. 2015. Pointer Networks. *ArXiv* abs/1506.03134 (2015).
[35] M. Vlachos, D. Gunopulos, and G. Kollios. 2002. Discovering similar multidimensional trajectories. *ICDE* (2002), 673–684.
[36] S. Wang and J. Jiang. 2016. Learning Natural Language Inference with LSTM. *ArXiv* abs/1512.08849 (2016).
[37] S. Wang and J. Jiang. 2017. Machine Comprehension Using Match-LSTM and Answer Pointer. *ArXiv* abs/1608.07905 (2017).
[38] Z. Wang, C. Long, G. Cong, and Y. Liu. 2020. Efficient and effective similar subtrajectory search with deep reinforcement learning. *PVLDB* 13 (2020), 2312 – 2325.
[39] F. Wu, T. Zhang, A. Souza, C. Fifty, T. Yu, and K. Q. Weinberger. 2019. Simplifying Graph Convolutional Networks. *ArXiv* abs/1902.07153 (2019).
[40] J. Xia, Y. Zhao, G. Liu, J. Xu, M. Zhang, and K. Zheng. 2019. Profit-driven Task Assignment in Spatial Crowdsourcing. In *IJCAI*. 1914–1920.
[41] M. Xie. 2014. EDS: a segment-based distance measure for sub-trajectory similarity search. *SIGMOD* (2014).
[42] C. Xiong, V. Zhong, and R. Socher. 2017. Dynamic Coattention Networks For Question Answering. *ArXiv* abs/1611.01604 (2017).
[43] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. Salakhutdinov, and Quoc V. Le. 2019. XLNet: Generalized Autoregressive Pretraining for Language Understanding. In *NeurIPS*.
[44] D. Yao, G. Cong, C. Zhang, and J. Bi. 2019. Computing Trajectory Similarity in Linear Time: A Generic Seed-Guided Neural Metric Learning Approach. *ICDE* (2019), 1358–1369.
[45] B. Yi, H. V. Jagadish, and C. Faloutsos. 1998. Efficient retrieval of similar time sequences under time warping. *ICDE* (1998), 201–208.
[46] A. W. Yu, D. Dohan, M. Luong, R. Zhao, K. Chen, M. Norouzi, and Quoc V. Le. 2018. QANet: Combining Local Convolution with Global Self-Attention for Reading Comprehension. *ArXiv* (2018).
[47] Y. Zhao, Y. Li, Y. Wang, H. Su, and K. Zheng. 2017. Destination-aware Task Assignment in Spatial Crowdsourcing. In *CIKM*. 297–306.
[48] Y. Zhao, S. Shang, Y. Wang, B. Zheng, Q. V. H. Nguyen, and K. Zheng. 2018. REST: A Reference-based Framework for Spatio-temporal Trajectory Compression. In *SIGKDD*. 2797–2806.
[49] Y. Zhao, J. Xia, G. Liu, H. Su, D. Lian, S. Shang, and K. Zheng. 2019. Preference-aware Task Assignment in Spatial Crowdsourcing. In *AAAI*. 2629–2636.
[50] Y. Zhao, K. Zheng, J. Guo, B. Yang, T. B. Pedersen, and C. S. Jensen. 2021. Fairness-aware Task Assignment in Spatial Crowdsourcing: Game-Theoretic Approaches. In *ICDE*.
[51] Y. Zhao, K. Zheng, Y. Li, H. Su, J. Liu, and X. Zhou. 2019. Destination-aware task assignment in spatial crowdsourcing: A worker decomposition approach. *TKDE* 32, 12 (2019), 2336–2350.
[52] Y. Zhaon, K. Zheng, Y. Cui, H. Su, F. Zhu, and X. Zhou. 2020. Predictive Task Assignment in Spatial Crowdsourcing: A Data-driven Approach. In *ICDE*. 13–24.
[53] K. Zheng, Y. Zhao, D. Lian, B. Zheng, G. Liu, and X. Zhou. 2020. Reference-Based Framework for Spatio-Temporal Trajectory Compression and Query Processing. *IEEE Transactions on Knowledge and Data Engineering* 32 (2020), 2227–2240.
[54] K. Zheng, Y. Zheng, N. J. Yuan, S. Shang, and X. Zhou. 2014. Online Discovery of Gathering Patterns over Trajectories. *IEEE Transactions on Knowledge and Data Engineering* 26 (2014), 1974–1988.